

Handoff: independent project in a shared repository

Repository: mayerspitz/tailored-travel-planning (temporary host) | Isolation: orphan branch | Future: its own repository

Read this first. You are starting a NEW, INDEPENDENT software project. It is only temporarily hosted in this GitHub repository and will later move to its own repository. The repository's main line of development (branch `main`) is a different product that has nothing to do with your project. Ignore everything on `main`: its README, CLAUDE.md, HANDOFF.md, SPEC.md, code, rules, brand and business. None of it applies to you.

0. Project details

The owner gives you the project details directly in the chat: name, purpose, scope, stack and priorities. Treat the chat as the source of truth for WHAT to build. This document only defines WHERE and HOW to work.

- Derive a short **slug** from the project name (lowercase, hyphens, e.g. "Acme Notes" becomes `acme-notes`). It is written as `<slug>` below.
- If anything essential is missing from the chat (name, purpose or stack), ask the owner before building.
- Record the details you were given in `STATUS.md` in your first commit, so later sessions do not depend on chat history.

1. Isolation model (non-negotiable)

Your project lives on an **orphan branch**: a branch with no shared history with `main`. This gives you a clean tree containing only your files, no chance of an accidental merge into the other product, and a one-command move to its own repository later, with full history (section 7).

1. Never merge, rebase or cherry-pick between your branches and `main` or any other product branch. Never open a pull request whose base is `main`.
2. Never modify files on `main` or on the other product's branches.
3. Share nothing at runtime: your own database, environment variables and secrets, hosting service and third-party accounts and keys. Never reuse the other product's `.env`, database or hosting services.
4. `main` contains a hosting blueprint (`render.yaml`) that deploys the other product. Do not add one to your branch unless it defines your OWN service, pinned to your branch, with names that cannot collide.
5. Do not name, describe or reference the other product anywhere in your project (code, docs, commits, package names). Your project must read as if it always lived in its own repository.
6. If your session opens on `main` and loads that branch's CLAUDE.md or other instructions, disregard them and switch to your branch (section 2).

2. Branches

- Trunk (your "main"): `project/<slug>`
- Work branches: `project/<slug>--<topic>`, created from the trunk and merged back into it by pull request with base `project/<slug>`.
- If your coding environment assigns you a fixed branch name (for example `claude/. . .`), create it from your trunk, or as an orphan in the first session. Never create it from `main`.

Creating the trunk (first session only)

```
git clone https://github.com/mayerspitz/tailored-travel-planning.git work
cd work
git checkout --orphan project/<slug>
git rm -rf . > /dev/null 2>&1
rm -rf .* ./.[!]* 2>/dev/null # empties the working tree, keeps .git
# create the skeleton from section 3, then:
git add -A
git commit -m "Initialize <project name>"
git push -u origin project/<slug>
```

Every later session

```
git fetch origin project/<slug>
git checkout project/<slug>
git pull
```

Confirm isolation before working: `git log --oneline | tail -1` must show your "Initialize" commit, and the tree must contain only your files.

3. Skeleton for the first commit

README.md	what it is, how to run, how to test
CLAUDE.md	rules for AI contributors to THIS project (write your own)
STATUS.md	resume point: goal, project details from chat, stage checklist, decisions, next steps
.gitignore	node_modules, dist, build output, .env*, logs, OS/editor files
.env.example	every environment variable with a comment (no real values)
<source dirs>	per your stack

Keep the project self-contained at the branch root, so moving it to its own repository needs no path rewriting.

4. Working discipline (sessions can end without warning)

- Commit and push after every meaningful step, not only at the end. Use small commits with imperative messages.
- Keep STATUS.md current at every push: the goal, what is done, what is in progress, what is next, and exactly how to resume. A new agent must be able to continue from the repository alone, with no chat history.
- Record decisions and their reasons in STATUS.md or docs/decisions.md.
- Never commit secrets, .env files, screenshots, build output or scratch files.
- After a session or container restart, run `git status` first, and commit any half-finished edits as "WIP" before anything else.

5. Quality bar

- Tests exist from the start and run offline, without credentials.
- Every external service call has a timeout, error handling and a clearly labeled fallback. A third-party outage must never crash the app.
- Build and tests pass before every push to the trunk.

6. Shared-repository etiquette

- Issues and pull requests in this repository are shared with the other product. Prefix every issue and pull request title with [`<slug>`].
- If you add CI (for example GitHub Actions), scope every workflow to your branches only, and never add workflows that run on `main`:

```
on:
  push:
    branches: ['project/<slug>', 'project/<slug>--*']
```

7. Moving to its own repository (when the owner decides)

1. The owner creates an empty repository, for example `<owner>/<slug>`.
2. Push the trunk as that repository's main branch, with full history:

```
git push https://github.com/<owner>/<slug>.git project/<slug>:main
```

- Push any open work branches the same way, set the default branch in the new repository, and point future sessions there.
- Delete the `project/<slug>*` branches from the old repository only after the owner confirms the new repository is complete.

8. First session checklist

- [] Project details received in chat and recorded in `STATUS.md` (section 0).
- [] Orphan trunk created and pushed (section 2).
- [] Skeleton committed (section 3), with the plan in `STATUS.md`.
- [] Your own `CLAUDE.md` written for this project.
- [] First feature started on a `project/<slug>--<topic>` branch.